

# Market Sentiment Classification of Financial Tweets

Text Mining — Spring Semester 2025/2026 · NOVA IMS

**Group 12** — João Cardoso (20240529) · Simon Sazonov (20221689) · Artem Polikarpov (20250443)

## 1 Data Exploration

The training corpus contains 9,543 financial tweets, each labelled **Bearish (0)**, **Bullish (1)** or **Neutral (2)**. The corpus is clean: it has no missing values, no empty strings and no duplicate tweets, so no rows had to be removed and duplicates cannot leak across data splits. The distributed test corpus contains 2,388 unlabelled tweets with an explicit id column, which is used directly as the identifier in `pred_12.csv`. The handout mentions 299 test rows, but the distributed file contains 2,388 and predictions cover all of them.

### 1.1 Class distribution

The three classes are strongly imbalanced, as shown in Figure 1: Neutral accounts for 64.7% of the tweets (6,178), Bullish for 20.2% (1,923) and Bearish for 15.1% (1,442). This imbalance shapes the rest of the project. A classifier that always predicted Neutral would already be correct on almost two thirds of the corpus while never identifying a single Bearish or Bullish tweet, which are precisely the classes a market participant acts on. Overall accuracy is therefore not a sufficient measure of quality on this corpus, and the evaluation must weight performance on the minority classes explicitly. In practical terms, all data splits and cross-validation are stratified so that the minority classes remain represented, classifiers that support it receive balanced class weights, and per-class results are inspected for every model.

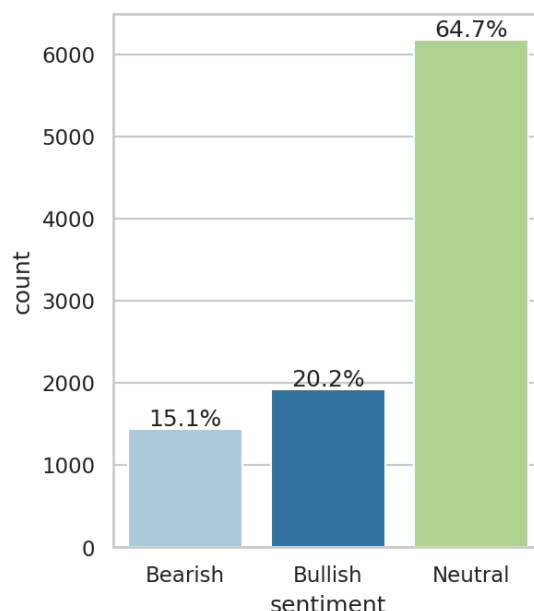


Figure 1: Distribution of sentiment classes in the training corpus.

### 1.2 Class vocabulary

Nineteen words, among them *stock(s)*, *market(s)*, *economy*, *china*, *trade*, *oil*, *price* and *earnings*, rank among the 50 most frequent words of all three classes simultaneously. They are general financial

vocabulary rather than sentiment carriers, and they were excluded from the visualisations so that class-specific vocabulary becomes visible. The exclusion applies only to the figures; models receive the full text.

As shown in Figure 2, the remaining vocabulary separates the classes clearly. Bearish tweets are dominated by miss and downgrade language (*misses, cut, lower, falls, downgraded*) together with crisis terms such as *coronavirus*; Bullish tweets mirror them with beat and upgrade language (*beats, raised, higher, buy, target, eps*); Neutral tweets use the factual vocabulary of the corporate calendar (*results, reports, declares, dividend, conference, transcript*). The bigrams in the same figure sharpen this picture: after removing eleven boilerplate pairs common to all classes (*stock market, wall street, seeking alpha*), the leading Bearish and Bullish bigrams turn out to be built on identical stems that differ only in the verb, such as *target cut* against *target raised* and *misses revenue* against *beats revenue*. Sentiment in this corpus is thus encoded in short word pairs, which motivates including n-grams in the Bag-of-Words feature space.

Two further consequences follow. The strong lexical separability suggests that a simple word-frequency model will already be competitive, giving the more expensive contextual models a concrete baseline to beat. And because company and data-vendor names (*marketscreener*) rank among the most frequent tokens, a classifier could end up learning company identities rather than sentiment; cashtags are therefore normalised to a generic ticker token during preprocessing.



Figure 2: Word clouds (left) and bigram clouds (right) for Bearish, Bullish and Neutral tweets, after excluding the vocabulary shared by all three classes.

### 1.3 Financial-domain signals

Finance-specific tokens that generic text-cleaning pipelines delete carry class signal in this corpus, as shown in Figure 3. Percentage expressions occur in 21.2% of Bullish and 16.3% of Bearish tweets but in only 3.1% of Neutral ones: directional moves such as “+5%” express an opinion about a stock, while neutral reporting rarely quantifies one. Cashtags lean Bullish (25.1%, against roughly 13% in the other classes), as do bare numbers (47.7% against 29.8% in Neutral). Neutral tweets are the most news-like, with the highest incidence of URLs (49.8%), hashtags (11.0%) and @mentions (3.8%).

A cleaning step that deletes every digit, % and \$ would therefore erase the strongest signal separating opinionated from neutral tweets. For this reason the preprocessing stage normalises tickers, percentages and numbers to placeholder tokens instead of deleting them, and keeps negation words that standard stop-word lists would remove.

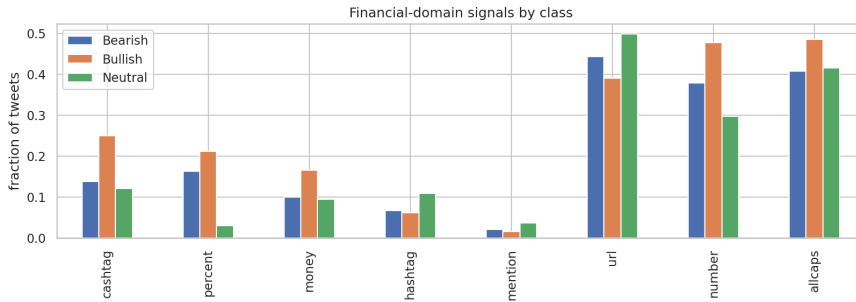


Figure 3: Fraction of tweets containing each finance-specific signal, by class.

## 1.4 Tweet length

Tweets are short and uniform: the median tweet has 11 whitespace tokens (mean 12.2, maximum 32), and per-class medians vary only between 11 and 12, so length itself does not separate the classes. The tail of the distribution sets the sequence-model configuration: the 95th percentile is 21 tokens and the 99th is 23, so a maximum sequence length of 32 word-tokens covers every tweet without truncation, and transformer tokenizers receive headroom above this value to absorb subword expansion.

## 2 Data Preprocessing

### 2.1 Corpus split

The labelled corpus was split into training, validation and internal test sets in a 70/15/15 ratio (6,679 / 1,432 / 1,432 tweets), stratified by class so that each subset preserves the class distribution to within 0.1 percentage points, with a fixed random seed for reproducibility. An internal test set is needed because the distributed `test.csv` is unlabelled and serves only to produce the submission file: without a held-out portion of the labelled data, the validation set would be used both to select models and to estimate final performance, and that estimate would be optimistically biased. The protocol is therefore: candidate models are compared and tuned on the validation set, the selected final model is scored once on the internal test set, and the submission model is retrained on all 9,543 labelled tweets before predicting `test.csv`.

The integrity checks (duplicate, null and empty-text removal) run before the split, because a duplicate tweet present in two subsets would let the model see part of its evaluation data during training; on this corpus the checks removed nothing, so the guarantee is structural. Every step that learns parameters from data — vectorizers, vocabularies, scalers and the classifiers themselves — is fit on the training subset only, inside scikit-learn pipelines, so the same leak-free behaviour holds during cross-validation.

### 2.2 Cleaning pipeline

Six preprocessing techniques were implemented and demonstrated individually in the tests notebook: regular-expression normalisation (URLs, mentions, cashtags, percentages and numbers), lowercasing, special-character removal, stop-word removal, lemmatisation and stemming. They compose into a single finance-aware cleaner whose design follows from the exploratory findings of the previous section rather than from generic defaults:

- **URLs and @mentions are removed.** Twitter-shortened links are opaque identifiers and handles are names; neither contributes sentiment vocabulary.
- **Financial tokens are normalised, not deleted.** Cashtags become ticker, percentage expressions become pct and bare numbers become num. This keeps the strongest opinionated-versus-neutral

signal found in the data (percentages appear in 21.2% of Bullish and 16.3% of Bearish tweets against 3.1% of Neutral) while collapsing thousands of distinct strings into three shared tokens. Replacing company tickers with a generic token also prevents a classifier from memorising company identities instead of sentiment vocabulary.

- **Special-character removal runs after the normalisation**, so the inserted placeholder tokens survive the letters-only step; lowercasing is applied first, and the placeholders are inserted already in lower case.
- **Stop words are removed with negation kept**. The standard NLTK stoplist contains *not*, *no* and *never*; deleting them can invert the meaning of a tweet (“not bullish”), so a negation list is subtracted from the stoplist before filtering.
- **Lemmatisation is preferred over stemming**. Both were implemented and compared on example sentences; lemmatisation maps inflections to dictionary forms (*shares* → *share*) while stemming truncates to non-words (*raised* → *rais*). Since later sections inspect the most discriminative features, readable dictionary forms were retained.

As an example, the tweet “\$ORLY strong up 11.7% into large volume pocket w/ room to \$360” becomes “*ticker strong pct large volume pocket w room num*”.

The cleaned text feeds the Bag-of-Words and word2vec representations only. Transformer encoders receive the raw text, because their subword tokenizers handle casing, punctuation and unknown words natively, and removing that surface information would discard cues those models exploit.

### 3 Feature Engineering

Three families of representations feed the classification stage, summarised in Table 1: sparse Bag-of-Words vectors and averaged word2vec embeddings computed on the cleaned text, and transformer sentence embeddings computed on the raw text. Every representation that learns parameters does so on training data only: vectorizer vocabularies and document-frequency weights are fit inside the model pipelines on training folds, and the word2vec model is trained on the training split.

| Representation                      | Dimensions     | Input        | Parameters learned from |
|-------------------------------------|----------------|--------------|-------------------------|
| Term counts / TF-IDF                | 4,600 (sparse) | cleaned text | training folds          |
| TF-IDF + bigrams                    | 9,104 (sparse) | cleaned text | training folds          |
| word2vec, averaged                  | 100 (dense)    | cleaned text | training split          |
| MiniLM sentence embeddings          | 384 (dense)    | raw text     | frozen, pretrained      |
| FinBERT — <i>Extra Work</i>         | 768 (dense)    | raw text     | frozen, pretrained      |
| twitter-RoBERTa — <i>Extra Work</i> | 768 (dense)    | raw text     | frozen, pretrained      |

Table 1: The representation families produced for the classification stage.

#### 3.1 Bag-of-Words

Three variations were implemented: raw term counts, TF-IDF weighting, and TF-IDF over unigrams and bigrams. With terms required to appear in at least two training tweets, the unigram vocabulary

contains 4,600 entries; adding bigrams roughly doubles it to 9,104, and the matrices remain extremely sparse (about 0.16% of entries are non-zero), a regime that linear models handle natively. A worked example in the tests notebook traces one tweet (“*WWE stock price target cut to \$57 from \$79 at Benchmark*”) through all three variants: raw counts store frequencies alone; TF-IDF shifts weight from corpus-common to rare words (*stock* receives 0.21 against 0.54 for *wwe*, although both occur once in the tweet); and the bigram variant adds word pairs with typically above-average weight, because pairs are rarer than the words they contain. All six mirror-verb bigrams identified in the exploratory analysis (*target cut/raised*, *miss/beat revenue*, *eps miss/beat*) survive cleaning and enter the vocabulary in lemmatised form, so the phrase-level sentiment signal is available to every model trained on this representation.

### 3.2 word2vec

A skip-gram word2vec model was trained on the 6,679 training tweets (100 dimensions, context window 5, minimum count 2, 10 epochs, single-threaded for reproducibility), and each tweet is represented as the mean of its in-vocabulary word vectors. Training on the project corpus alone keeps the representation leak-free but exposes the limits of 6,679 short texts. The model does learn the earnings cluster — the nearest neighbours of *beat* include *miss*, *eps* and *revenue* — yet *beat* and *miss* are each other’s closest neighbours: antonyms occur in the same contexts, so distributional similarity does not encode sentiment polarity, and averaging word vectors may blur exactly the Bearish/Bullish distinction this task requires. Neighbourhoods of rarer words are noisy (*bullish* neighbours include *burn* and *crazy*), which is consistent with the small corpus. This sets a concrete expectation for the evaluation: averaged word2vec should be the weakest family, and pretrained encoders should recover what the small corpus cannot provide.

### 3.3 Transformer encoders

Sentence embeddings were extracted with the all-MiniLM-L6-v2 sentence-transformer (384 dimensions). The encoder is frozen, so this is feature extraction rather than fine-tuning, and it receives raw text for the reasons given in the preprocessing section. Embeddings for all splits are computed once and cached; encoding runs on GPU when available and falls back to CPU.

**Extra Work.** Two additional transformer encoders beyond the required one were applied, chosen to match the domain from complementary angles: FinBERT, pretrained on financial text (the content), and twitter-RoBERTa, pretrained on tweets (the register). For both, a tweet is represented by the mean of the last hidden state over its real tokens, giving 768-dimensional frozen embeddings.

Encoded under every representation, the same example tweet shows the two regimes side by side: the TF-IDF row activates 14 named features out of 9,104, each readable on its own (*target cut* among them), while the dense families produce vectors in which every coordinate is non-zero and individually meaningless — 100, 384 or 768 dimensions whose information lies in the distances between tweets rather than in any single value. The two regimes therefore offer the classifiers different evidence, explicit lexical features against pretrained semantic context, and the classification stage measures which carries more sentiment signal.